ASSESSMENT - 2

NAME: M. Rumana khan

REG NO: 192471025

COURSE CODE / NAME: CSA0610 - Design and Analysis Of Algorithms

---

Assignment Title: Debugging & Optimisation Task : SOC Alert-Triage Module of “SentinelOps”.

Introduction

This submission analyses and repairs the alert-triage module of SentinelOps, the Security Operations Centre of a cloud-services company. The module sorts alerts by severity, checks source IP addresses against a blocklist, finds attack signatures in log lines and finds the two closest login locations. It passed code review but failed in production, and it must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day. Part (a) reproduces the failures and identifies the nine defects, Part (b) explains their root causes and corrects them one function at a time, Part (c) optimises the corrected module, Part (d) compares the module before and after by derivation and measurement, and Part (e) presents the final tested code.

Part (a) – Reproduce and identify the defects (10 marks)

Running the original module on the test data gives a wrong answer in all four functions, and nine defects explain every failure.

The original module was saved as `buggy.py` and was run on the test data with the driver below, which was appended at the bottom of the file.

```
if __name__ == "__main__":

    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
```

```
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]

queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]

log, pattern = "FAILFAIXFAIL", "FAIL"

events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]


print("sort_alerts    :", sort_alerts(list(alerts)))

print("is_blocked     :", [(q, is_blocked(blocklist, q)) for q in queries])

print("find_signature :", find_signature(log, pattern))

print("closest_logins :", closest_logins(events))
```

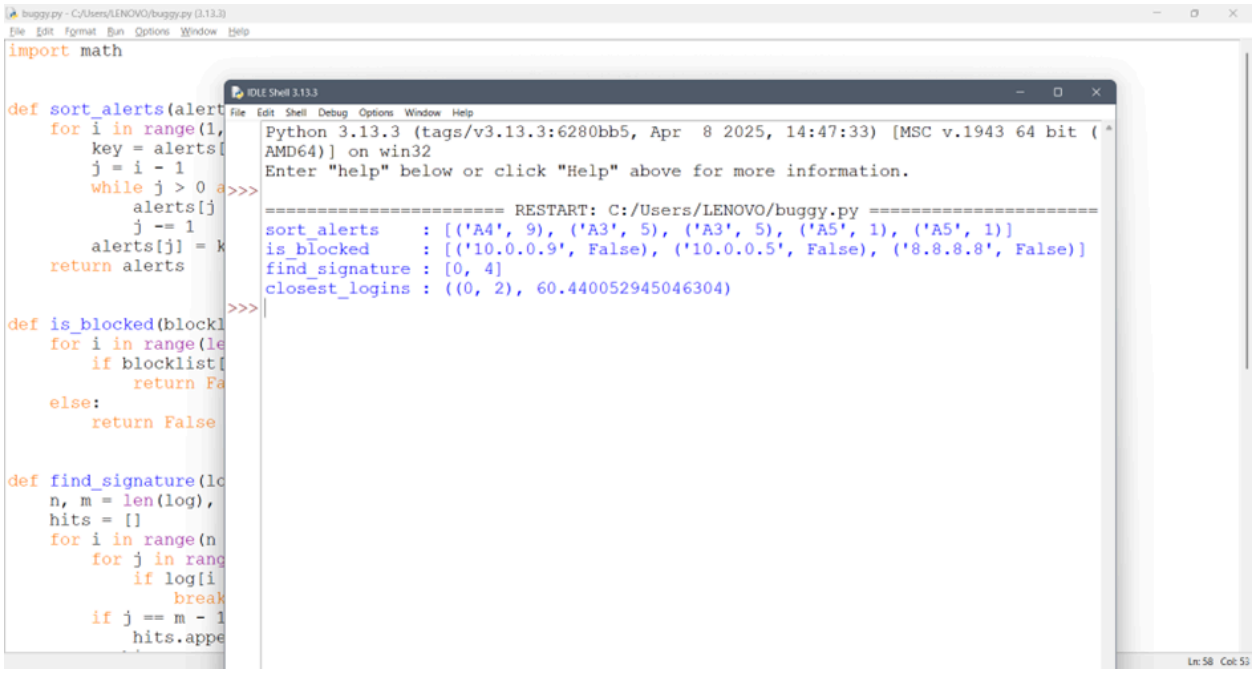


Table 1 – Expected versus actual output of the original code

Function	Expected output	Actual output (original code)	What went wrong
sort_alerts	A2:9, A4:9, A3:5, A1:3, A5:1	A4:9, A3:5, A3:5, A5:1, A5:1	A1 and A2 vanished; A3 and A5 appear twice
is_blocked	True, True, False for the three queries	False, False, False	Blocked IPs get through
find_signature	[0, 8]	[0, 4]	Real hit at 8 missed; false hit at 4 (`FAIX`)
closest_logins	pair (0, 5), distance 1.414 km	pair (0, 2), distance 60.44 km	Reports the farthest pair, not the closest

Table 2 – Defects identified in the original code

S.No	Function	Faulty line	Nature of the error
1	sort_alerts	while j > 0 and alerts[j][1] < key[1]:	Off-by-one boundary: index 0 is never compared, so a key can never move past `alerts[0]`
2	sort_alerts	alerts[j] = key	Off-by-one insertion point: the gap is at `j + 1`, so the key overwrites a live element
3	is_blocked	return False inside if blocklist[i] == ip:	Inverted logic: a match is reported as not blocked
4	find_signature	for i in range(n - m):	Off-by-one: the last window starts at `n - m`, which `range` excludes
5	find_signature	if j == m - 1: after the inner loop	Wrong success test: a break at the last character also leaves `j == m - 1`, so a mismatch is recorded as a match
6	closest_logins	min_dist = 0	Wrong starting value: with 0 no distance can ever be smaller, so

			the minimum search cannot work
7	closest_logins	if d > min_dist:	Wrong comparison: keeps the largest distance, so it finds the farthest pair
8	closest_logins	for j in range(len(events)):	Includes self pairs (i, i) with distance 0, which would always win once defects 6 and 7 are fixed
9	closest_logins	same inner loop	Includes mirrored pairs (j, i) as well as (i, j), so every pair is measured twice

Defects 8 and 9 sit on the same line but are different faults (self pairs versus repeated pairs); counting them separately gives the nine the brief states. The `for ... else` in `is_blocked` and the strict `<` in `sort_alerts` are correct and stay (the strict `<` is what keeps equal severities in arrival order).

Part (b) – Root causes and systematic debugging (12 marks)

Each defect has one root cause, and several hide each other, so the fixes were applied one function at a time with a re-test after every change.

Table 3 – Root cause, fix and verification for each defect

#	Root cause and how it produced the wrong output	Fix	Verified by
1	The guard `j > 0` stops one step early, but index 0 is a valid slot. A key larger than `alerts[0]` is never compared with it, so it cannot reach the front	`while j >= 0 and …`	Trace below: A2:9 now reaches index 0
2	When the loop ends, `j` is the last element that is not smaller than the key (or -1), so the free slot is `j + 1`. Writing to `alerts[j]` overwrites a live element: A1 is lost and A3, A5 are duplicated	`alerts[j + 1] = key`	Trace below; all five alerts survive
3	The success branch returns `False`, so a match is reported as not blocked and every query ends as False	`return True` on a match	10.0.0.9 and 10.0.0.5 give True, 8.8.8.8 gives False

4	`range(n - m)` ends at `n - m - 1`, but a pattern may start at `n - m`. The window at the very end of the log is never tried, so the hit at 8 is lost	`range(n - m + 1)`	`find_signature("xxFAIL", "FAIL")` gives [2]
5	After a `break` the variable `j` still holds the failing index. When the failing character is the last one, `j == m - 1` is true and the window is recorded as a match. `FAIX` breaks at j = 3 and is wrongly reported at 4	`for … else`: append only when the loop finished without a break	Alignment table below; `find_signature("FAIX", "FAIL")` gives []
6	`min_dist = 0` means no distance can ever be smaller than the current best, so a minimum search cannot start	`min_dist = math.inf`	Step 6 in the re-test table
7	`d > min_dist` keeps the largest distance, so the function reports the farthest pair, (0, 2) at 60.44 km	`d < min_dist`	Step 6 in the re-test table
8	`j` starts at 0, so the pair (i, i) with distance 0 is examined	`range(i + 1, n)`	Step 7: result is (0, 5), not (0, 0)

	and would always win after fixes 6 and 7		
9	The same loop also visits (j, i) after (i, j): 30 mirrored distances (plus 6 self pairs) for 6 events instead of 15	`range(i + 1, n)`	Step 7; 15 distances counted

Table 4 – Insertion trace for the alerts list (each row is the list after that outer iteration):

i	key	Original code	Corrected code
1	A2:9	A2:9, A2:9, A3:5, A4:9, A5:1 (A1 overwritten)	A2:9, A1:3, A3:5, A4:9, A5:1
2	A3:5	A2:9, A3:5, A3:5, A4:9, A5:1	A2:9, A3:5, A1:3, A4:9, A5:1
3	A4:9	A4:9, A3:5, A3:5, A3:5, A5:1 (A2 overwritten)	A2:9, A4:9, A3:5, A1:3, A5:1
4	A5:1	A4:9, A3:5, A3:5, A5:1, A5:1	A2:9, A4:9, A3:5, A1:3, A5:1

At i = 3 the corrected code shifts A1:3 and A3:5 right, then stops at A2:9 because 9 < 9 is false, so A4 lands after A2 and equal severities keep arrival order.

Table 5 – Alignment of pattern `FAIL` against log `FAILFAIXFAIL` (n = 12, m = 4):

Start i	Window	Correct result	Original code
0	FAIL	match	match
1, 2, 3	AILF, ILFA, LFAI	no match	no match
4	FAIX	no match (X ≠ L at j = 3)	false match, because the break left j = 3 = m - 1
5, 6, 7	AIXF, IXFA, XFAI	no match	no match
8	FAIL	match	never tried: `range(12 - 4)` stops at 7

Table 6 – Re-test after every correction (each row adds one change to the original):

Step	Change applied	Result of the re-test
1	Fix 2 only	A1:3, A2:9, A4:9, A3:5, A5:1 – nothing lost or duplicated, but A1 is stuck in front
2	Add fix 1	A2:9, A4:9, A3:5, A1:3, A5:1 – correct (fix 1 alone is worse: A1:3 four times, so the two defects hide each other)
3	Fix 3	True, True, False – correct
4	Fix 4 only	[0, 4, 8] – end hit found, false hit at 4 remains
5	Add fix 5	[0, 8] – correct
6	Fix 6 and fix 7	((0, 0), 0.0) – the self pair now wins, which exposes defect 8
7	Add fixes 8 and 9	((0, 5), 1.414) – correct

The corrected module, `fixed_only.py`, contains only the nine fixes; each is marked with a `# FIX n` comment that matches the numbering in Table 2.

```
"""Part (b): the ORIGINAL module with only the nine defects corrected.


Each fix is marked  # FIX n  so it can be traced back to the defect table.

No restructuring or optimisation is done here (that is Part (c)).

"""

import math

def sort_alerts(alerts):  # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:   # FIX 1: j >= 0 (index 0 must be compared)
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key                    # FIX 2: drop key into the gap at j + 1
    return alerts


def is_blocked(blocklist, ip):  # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return True                        # FIX 3: a match means BLOCKED
    return False                               # searched everything: not on the list
```

```
def find_signature(log, pattern):  # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    if m == 0 or m > n:                    # boundary guard (empty / too-long pattern)
        return hits
    for i in range(n - m + 1):             # FIX 4: last start index is n - m (inclusive)
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:                              # FIX 5: for-else -> only runs if NO mismatch
            hits.append(i)
    return hits


def closest_logins(events):  # brute-force closest pair of login locations
    if len(events) < 2:                    # boundary guard (fewer than two events)
        return None, None
    min_dist = math.inf                    # FIX 6: start at +infinity, not 0
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):        # FIX 8 + FIX 9: j > i removes self (i,i) and mirrored (j,i) pairs
```

```
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < min_dist:                    # FIX 7: closest means SMALLEST, so '<'
                min_dist = d
                pair = (i, j)
    return pair, min_dist


if __name__ == "__main__":
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
    log, pattern = "FAILFAIXFAIL", "FAIL"
    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]

    print("sort_alerts    :", sort_alerts(list(alerts)))
    print("is_blocked     :", [(q, is_blocked(blocklist, q)) for q in queries])
    print("find_signature :", find_signature(log, pattern))
    print("closest_logins :", closest_logins(events))
    print("-- boundary tests --")
    print("sort equal sev :", sort_alerts([("E1", 5), ("E2", 5), ("E3", 5)]))
    print("sort empty/one :", sort_alerts([]), sort_alerts([("X", 1)]))
```

```
print("blocked empty  :", is_blocked([], "1.1.1.1"))
print("sig at end     :", find_signature("xxFAIL", "FAIL"))
print("sig longer     :", find_signature("ab", "abcd"))
print("sig == log     :", find_signature("FAIL", "FAIL"))
print("sig empty pat  :", find_signature("abc", ""))
print("closest 1 / 0  :", closest_logins([(1, 1)]), closest_logins([]))
print("closest dup pts:", closest_logins([(1, 1), (9, 9), (1, 1)]))
```

Output of the corrected module:

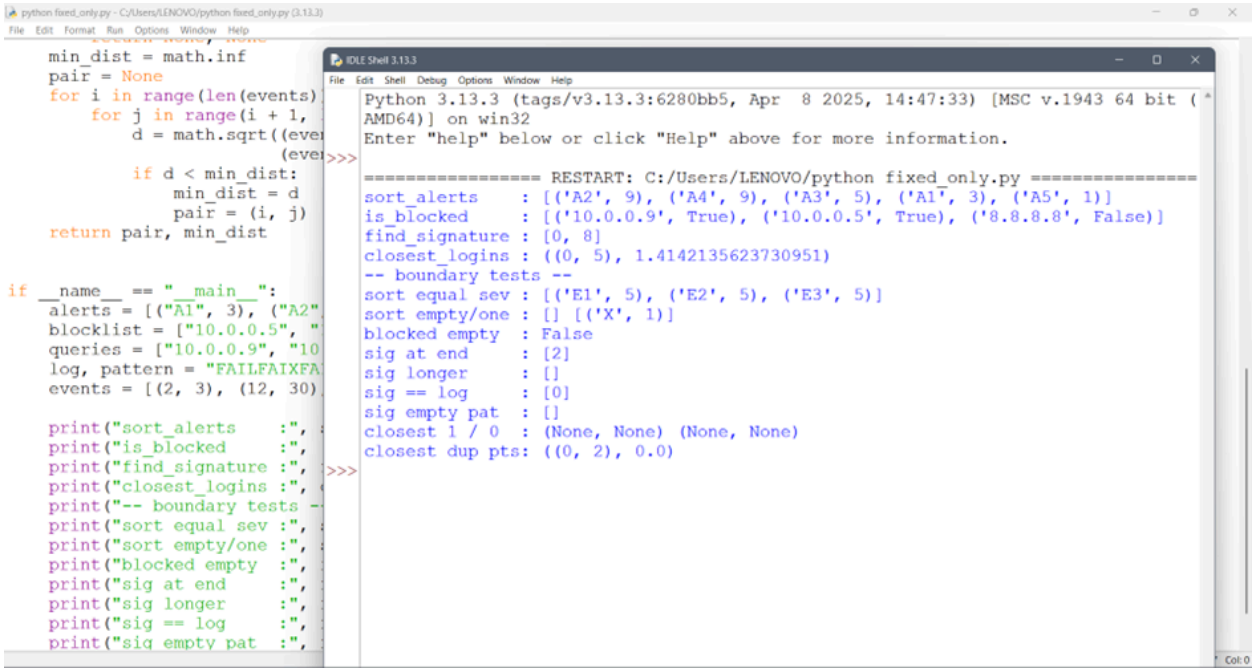


Table 7 – Boundary tests, original versus corrected code

Boundary case	Original code	Corrected code
Equal severities E1:5, E2:5, E3:5	E2:5, E3:5, E3:5 (E1 lost, E3 duplicated)	E1:5, E2:5, E3:5 (arrival order kept)
Empty list, single alert	[] and [X:1] (already correct)	[] and [X:1]
Empty blocklist	False (already correct)	False
Pattern at the very end (`xxFAIL`)	[]	[2]
Pattern equals the log (`FAIL`)	[]	[0]
Pattern longer than the log (`ab`, `abcd`)	[] (by luck of a negative range)	[] (explicit guard)
Empty pattern	Crashes with `UnboundLocalError` on `j`	[] (explicit guard)

Fewer than two login events	(None, 0) – claims a distance of 0	(None, None)
Duplicate location (1,1) twice	(0, 1) at 11.3 km – wrong	(0, 2) at 0.0 km

The guards for an empty pattern and for fewer than two events are extra hardening and are not among the nine counted defects.

Part (c) – Optimising the corrected module (10 marks)

Four changes reduce the work without changing any output, and the largest saving is in `closest_logins`, which drops from 31,996,000 to 13,746 distance computations for 8,000 events. The final code is in Part (e) and the measurements are in Part (d).

Table 8 – Summary of optimisations

Function	Before (corrected)	After	Why the output is unchanged
`sort_alerts`	Insertion sort, $O(n^2)$ worst case	Stable built-in Timsort with `reverse=True`, O(n log n) worst case	Both are stable, so equal severities keep arrival order
`is_blocked`	Scan the whole list on every query, O(k)	Hash set built once, O(1) average per query	Set membership gives the same True/False as a scan

`find_signature`	Brute force with a post-loop test per window	Brute force with `for … else` and early guards	Same windows, same matches
`closest_logins`	All n(n-1)/2 pairs, one `sqrt` each	x-sorted sweep with pruning, squared distances, one `sqrt`	Pruned pairs cannot beat the best pair; ties resolve to the same pair

Justification of each optimisation

1. Sorting alerts. Insertion sort costs about n + I comparisons, where I is the number of out-of-order pairs, so on nearly sorted alerts it is excellent: 1,009 comparisons for 1,000 alerts in the measurements. It stays suitable for small batches or input that is guaranteed to be almost in order. It is not suitable as the production default, because a batch of 50,000 alerts in arbitrary order needs about $n^2/4$ = 625,000,000 comparisons. Python's built-in sort is stable, takes about n comparisons on already-ordered runs, and never exceeds O(n log n). `reverse=True` keeps equal items in their original order, so the requirement "equal severities in arrival order" still holds. If severities are small whole numbers, a counting sort (O(n + k)) is another valid option.

2. Repeated blocklist lookups. Convert the blocklist to a set once with `build_blocklist` (O(k)), then every lookup is a hash probe, O(1) on average. For a 5,000-entry blocklist and 50,000 lookups the sequential scan needed 187,233,115 comparisons; the set needed 50,000 probes. A sorted list with binary search (O(log k)) is the fallback if hashing is not allowed.

3. Signature scan. Only the n - m + 1 windows that can contain the pattern are tried; every window stops at its first mismatch; and `for … else` removes the old per-window test `j == m - 1`, so no character is compared twice and no comparison happens after a mismatch. An empty pattern or one longer than the log

returns before any comparison. Brute force still costs (n - m + 1)·m in the worst case (990,100 comparisons for `a`×10,000 against `a`×99 + `b`), and about n in ordinary text (99,997 for 100,000 characters). Going below that needs KMP or Boyer–Moore, which is beyond the brute-force technique assessed here.

4. Closest pair. Three savings, all visible in the code in Part (e):

- Self and repeated pairs are gone: only pairs with j after i are examined, so n(n-1)/2 pairs instead of n².
- Squared distances are compared, because the smallest square belongs to the smallest distance; `sqrt` runs once at the end instead of once per pair.
- Pruning: events are visited in order of x. For a fixed event, the loop stops as soon as the x-gap alone is larger than the best distance found so far, since the full distance can only be larger still. The test is `>` rather than `>=`, so pairs that tie for closest are never skipped and the tie-break matches brute force.

On the sample data the x order is events 0, 5, 3, 1, 4, 2. The sweep computes only two distances, (0, 5) with squared distance 2 and (1, 4) with 400, then stops every inner loop early, against 15 pairs and 15 square roots before. The pruning depends on the x spread: if every login has the same x, nothing is pruned and the cost returns to n(n-1)/2 (measured in Part (d)).

Part (d) – Before and after analysis (10 marks)

The measurements agree with the analysis: counts match the derived formulas, and the optimised `closest_logins` uses 2,328 times fewer distance computations than the corrected brute force at 8,000 events.

Table 9 – Operation counts and time complexity (n = alerts or events, k = blocklist size, Q = lookups, m = pattern length):

Function	Basic operation	Original code	Corrected, not optimised	Optimised
`sort_alerts`	Severity comparisons	Same loop shape, $O(n^2)$	Best n - 1 (already in order), worst n(n-1)/2 (reverse order), average about $n^2/4$: $O(n^2)$	Timsort: about n on ordered input, O(n log n) worst
`is_blocked`	Equality tests	Same cost as corrected (it exits at the first match, but returns the wrong value)	Hit at position p costs p + 1, miss costs k, so Q lookups cost $O(Q \cdot k)$	Build once O(k), then O(1) average per lookup: O(k + Q)
`find_signature`	Character comparisons	$(n - m) \cdot m$ at most, plus n - m post-loop tests, and the last window is never tried	Between n - m + 1 and $(n - m + 1) \cdot m$: $O(n \cdot m)$	Same bound, zero extra tests, about n on ordinary text
`closest_logins`	Distance computations (each has 1 `sqrt`)	n^2 (n self pairs, every pair twice)	n(n-1)/2, with n(n-1)/2 square roots: $O(n^2)$	Sort O(n log n) plus a pruned sweep, one `sqrt`;

worst case still $O(n^2)$

Derivations. Insertion sort: item i is compared with between 1 and i earlier items, so the total is n - 1 for ordered input, the sum of i = n(n-1)/2 for reversed input, and about half of that on average. Closest pair: the loop pair (i, j > i) runs the sum of (n - 1 - i) = n(n-1)/2 times, which is 31,996,000 for n = 8,000 against n^2 = 64,000,000 originally. For uniformly spread logins the closest distance is about the area's width divided by n, so each event has O(1) candidates inside the x-gap and the sweep is close to linear after the sort; that is an estimate, which the measurement below confirms.

The measurements were produced by the script `analysis.py`, listed at the end of this part, which counts operations directly in the same loops as the functions under test.

Table 10 – Closest pair: distance computations (uniform random logins in a 1,000 × 1,000 km area, fixed seed):

Events n	Original n^2	Corrected brute force	Optimised	Reduction	Seconds, brute force	Seconds, optimised
1,000	1,000,000	499,500	1,778	281×	0.22	0.001
2,000	4,000,000	1,999,000	3,794	527×	0.97	0.002
4,000	16,000,000	7,998,000	4,833	1,655×	4.39	0.005

8,000	64,000,000	31,996,000	13,746	2,328×	16.73	0.009

Worst case for the pruning: with 2,000 events that all share the same x, the optimised version performs 1,999,000 computations, the same as brute force. Both versions returned the identical pair at every size.

Table 11 – Insertion sort versus Timsort: severity comparisons (nearly sorted = already in order, then 1% of neighbouring pairs swapped):

Alerts n	**Input**	**Insertion sort**	**Timsort**	**$n^2/4$**
1,000	nearly sorted	1,009	1,850	250,000
1,000	random	251,543	8,626	250,000
5,000	nearly sorted	5,049	6,950	6,250,000
5,000	random	6,274,172	55,081	6,250,000
10,000	nearly sorted	10,095	14,118	25,000,000

10,000	random	24,873,462	120,144	25,000,000

Table 12 – Blocklist and signature measurements

Test	Before	After
50,000 lookups, 4,999-IP blocklist, half of them hits	187,233,115 comparisons, 2.11 s (list scan)	50,000 hash probes, 0.002 s (set)
`FAILFAIXFAIL` / `FAIL`	9 windows, 18 character comparisons	same 18, and no post-loop tests
100,000 random characters / 4-character pattern	99,997 windows, 99,997 comparisons	same
`a`×10,000 / `a`×99 + `b` (worst case)	9,901 windows, 990,100 comparisons ($n \cdot m$ = 1,000,000)	same: brute force keeps its worst case

Do the measurements agree with the analysis? Yes, with two nuances worth stating. First, every count lands on its formula: the brute-force pair count is exactly $n(n-1)/2$, random-input insertion sort is within 1% of $n^2/4$ (251,543 against 250,000 at n = 1,000), nearly sorted input costs about n plus the number of swapped pairs (1,009 for 1,000 alerts), and Timsort on random input is close to $n \cdot \log_2 n$ (120,144 against 132,877 at n = 10,000). Second, insertion sort actually used fewer comparisons than Timsort on nearly sorted input (1,009 against 1,850), which confirms that insertion sort is a sound choice there; Timsort's advantage is the

guaranteed O(n log n) bound on arbitrary input, where insertion sort needs about 625,000,000 comparisons for 50,000 alerts. For `closest_logins` the saving grows with n because the optimised count grows roughly in step with n while brute force grows with n^2, but the all-equal-x test shows the gain depends on the data and the worst case stays $O(n^2)$. The signature scan shows no change in comparisons because the corrected brute force was already minimal; its gain is correctness and the removed post-loop tests. Timings are from one machine and only indicate the trend; the counts are the reliable evidence.

```
"""Part (d) measurements. Run:  python analysis.py   (under a minute)


Counts are taken with explicit counters placed on the SAME loop structure as

the functions in sentinelops.py / fixed_only.py, so they measure the real

algorithms rather than a formula.

"""

import math

import random

import time


from sentinelops import closest_logins, closest_logins_bruteforce



# ---------------- closest pair: distance computations ----------------------

def count_original_buggy(n):

    """Original code evaluates every ordered pair incl. self pairs: n*n distances."""
```

```
    return n * n



def count_bruteforce(events):
    """Corrected brute force, instrumented: one distance per unordered pair."""
    cnt = 0
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            cnt += 1
    return cnt



def count_optimised(events):
        """Optimised sweep, same loop as closest_logins, counting distance evaluations."""
    n = len(events)
    order = sorted(range(n), key=lambda k: events[k][0])
    best_sq, cnt = math.inf, 0
    for a in range(n - 1):
        xi, yi = events[order[a]]
        for b in range(a + 1, n):
```

```
            xj, yj = events[order[b]]
            dx2 = (xj - xi) ** 2
            if dx2 > best_sq:
                break
            cnt += 1
            d2 = dx2 + (yj - yi) ** 2
            if d2 < best_sq:
                best_sq = d2
    return cnt



def closest_pair_table():
    rng = random.Random(42)
     print("\n== closest_logins: distance computations (uniform random, 1000 x 1000 km) ==")
    print(f"{'events n':>9} {'original':>12} {'brute (fixed)':>14} {'optimised':>10} "
          f"{'saving':>8} {'t_brute s':>10} {'t_opt s':>8}")
    for n in (1000, 2000, 4000, 8000):
        ev = [(rng.uniform(0, 1000), rng.uniform(0, 1000)) for _ in range(n)]
        brute = count_bruteforce(ev)
        opt = count_optimised(ev)
```

```
                t0 = time.perf_counter(); r1 = closest_logins_bruteforce(ev); t1 = time.perf_counter()
        r2 = closest_logins(ev); t2 = time.perf_counter()
        assert r1 == r2, "outputs differ!"
        print(f"{n:>9} {count_original_buggy(n):>12,} {brute:>14,} {opt:>10,} "
              f"{brute / opt:>7.0f}x {t1 - t0:>10.2f} {t2 - t1:>8.3f}")
    # worst case for the pruning: every login has the same x
    n = 2000
    ev = [(500.0, rng.uniform(0, 1000)) for _ in range(n)]
    print(f"\nWorst case for pruning (all x equal, n={n}): optimised = "
          f"{count_optimised(ev):,} vs brute = {count_bruteforce(ev):,}")



# ---------------- insertion sort: comparisons ------------------------------
def insertion_comparisons(sev):
    a = list(sev)
    cmp = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
```

```
            cmp += 1                    # one evaluation of a[j] < key
            if a[j] < key:
                a[j + 1] = a[j]
                j -= 1
            else:
                break
        a[j + 1] = key
    return cmp



class Counted:
    n = 0
    __slots__ = ("v",)

    def __init__(self, v):
        self.v = v

    def __lt__(self, other):
        Counted.n += 1
        return self.v < other.v
```

```
def timsort_comparisons(sev):
    Counted.n = 0
    items = [Counted(v) for v in sev]
    items.sort(reverse=True)
    return Counted.n



def sort_table():
    rng = random.Random(7)
    print("\n== sort_alerts: comparisons, severity descending ==")
    print(f"{'n':>7} {'input':>14} {'insertion':>12} {'Timsort':>10} {'n^2/4':>12}")
    for n in (1000, 5000, 10000):
        rand = [rng.randint(1, 1_000_000) for _ in range(n)]
        near = sorted(rand, reverse=True)          # already in the right order ...
        for _ in range(n // 100):                  # ... then 1% adjacent swaps
            k = rng.randrange(n - 1)
            near[k], near[k + 1] = near[k + 1], near[k]
        for name, data in (("nearly sorted", near), ("random", rand)):
            print(f"{n:>7} {name:>14} {insertion_comparisons(data):>12,} "
                  f"{timsort_comparisons(data):>10,} {n * n // 4:>12,}")
```

```
    print("n = 50,000 random: insertion sort would need about "
          f"{50000 * 50000 // 4:,} comparisons (n^2/4).")



# ---------------- blocklist lookups -------------------------------------

def blocklist_table():
    rng = random.Random(11)
    size, queries_n = 5000, 50000
    bl = [f"10.{rng.randrange(256)}.{rng.randrange(256)}.{rng.randrange(256)}" for _ in range(size)]
    bl = list(dict.fromkeys(bl))
    pos = {ip: k for k, ip in enumerate(bl)}
    qs = [rng.choice(bl) if rng.random() < 0.5 else
          f"192.168.{rng.randrange(256)}.{rng.randrange(256)}" for _ in range(queries_n)]
    # comparisons by sequential search: position + 1 on a hit, len on a miss
    lin = sum(pos[q] + 1 if q in pos else len(bl) for q in qs)
    t0 = time.perf_counter()
    for q in qs:
        q in bl
    t1 = time.perf_counter()
    s = frozenset(bl)
```

```
    for q in qs:
        q in s
    t2 = time.perf_counter()
     print("\n== is_blocked: blocklist of %d IPs, %d lookups (50%% hits) ==" % (len(bl), queries_n))
    print(f"sequential: {lin:,} comparisons, {t1 - t0:.3f} s   |   "
          f"hash set: {queries_n:,} lookups (~1 probe each), {t2 - t1:.4f} s")

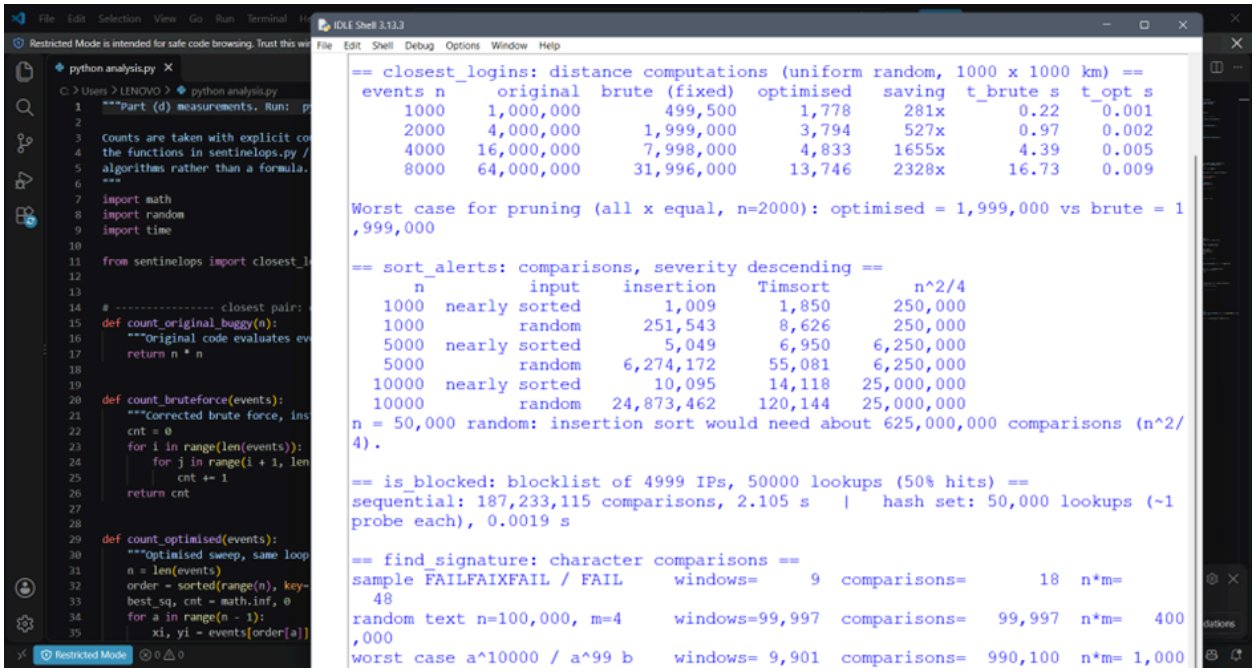


# ---------------- signature search: character comparisons ------------------
def sig_compare_count(log, pat):
    n, m = len(log), len(pat)
    if m == 0 or m > n:
        return 0, 0
    cmp = 0
    for i in range(n - m + 1):
        for j in range(m):
            cmp += 1
            if log[i + j] != pat[j]:
                break
    return cmp, n - m + 1
```

```
def signature_table():
    print("\n== find_signature: character comparisons ==")
    for label, log, pat in (
        ("sample FAILFAIXFAIL / FAIL", "FAILFAIXFAIL", "FAIL"),
         ("random text n=100,000, m=4", "".join(random.Random(5).choice("abcdefgh") for _ in range(100000)), "abcd"),
        ("worst case a^10000 / a^99 b", "a" * 10000, "a" * 99 + "b"),
    ):
        c, w = sig_compare_count(log, pat)
            print(f"{label:<30} windows={w:>6,}  comparisons={c:>9,}  n*m={len(log) * len(pat):>10,}")



if __name__ == "__main__":
    closest_pair_table()
    sort_table()
    blocklist_table()
    signature_table()
```



Part (e) – Production-quality code and submission (8 marks)

The final module and its test harness are listed below, and all 25 tests pass.

Listing 1 – sentinelops.py (the _insertion and _bruteforce functions are the corrected baselines that the tests and analysis.py compare against):

"""SentinelOps SOC alert-triage module (corrected and optimised).

Functions

---

sort_alerts sort alerts by severity, highest first, ties in arrival order

build_blocklist one-off conversion of a blocklist into a hash set

is_blocked check whether a source IP is on the blocklist

find_signature every start position of an attack signature in a log line

closest_logins   closest pair of login locations (pruned brute force)

Reference baselines kept for validation and for the measurements in

analysis.py: sort_alerts_insertion and closest_logins_bruteforce.

"""

```
import math



# ---------------------------------------------------------------------------
# 1. Sorting alerts by severity
# ---------------------------------------------------------------------------
def sort_alerts_insertion(alerts):
    """Corrected insertion sort (baseline): severity descending, stable.

    Strict '<' keeps equal severities in arrival order. O(n^2) worst case.
    """
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
```

```
            j -= 1
        alerts[j + 1] = key
    return alerts



def sort_alerts(alerts):
    """Sort alerts in place by severity (item[1]), highest first.

    Alerts of equal severity keep their arrival order. Uses Python's Timsort,
    which is stable, O(n log n) worst case and ~O(n) on nearly sorted input.
    `reverse=True` preserves the original order of equal keys.
    """
    alerts.sort(key=lambda alert: alert[1], reverse=True)
    return alerts



# ---------------------------------------------------------------------
# 2. Blocklist lookups
# ---------------------------------------------------------------------
def build_blocklist(blocklist):
    """Return a frozenset of blocked IPs for O(1) average-time lookups.
```

```
    Build it ONCE, then reuse it for every query.
    """
    return frozenset(blocklist)


def is_blocked(blocklist, ip):
    """Return True if `ip` is on the blocklist, otherwise False.

    Works with a list (linear scan, stops at the first match) or with the set
    returned by build_blocklist (hash lookup). An empty blocklist gives False.
    """
    return ip in blocklist
# ---------------------------------------------------------------------------
# 3. Signature search
# ---------------------------------------------------------------------------
def find_signature(log, pattern):
    """Return every start index at which `pattern` occurs in `log`.

    Brute-force matching. Only the n - m + 1 windows that can hold the pattern
    are tried, each window stops at its first mismatch, and no comparison is
```

```
    repeated after a mismatch (for/else replaces the old post-loop test).
    An empty pattern or one longer than the log returns [].
    """
    n, m = len(log), len(pattern)
    if m == 0 or m > n:
        return []
    hits = []
    for i in range(n - m + 1):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:                   # no mismatch in the whole window
            hits.append(i)
    return hits

# ---------------------------------------------------------------------------
# 4. Closest pair of login locations
# ---------------------------------------------------------------------------
def closest_logins_bruteforce(events):
    """Corrected brute-force closest pair (baseline): every pair once, O(n^2)."""
    if len(events) < 2:
        return None, None
```

```
    best, pair = math.inf, None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < best:
                best, pair = d, (i, j)
    return pair, best
def closest_logins(events):
    """Return ((i, j), distance) for the two closest login locations, i < j.

    Same result as closest_logins_bruteforce, with three savings:
      * each unordered pair is examined once (no self or mirrored pairs);
      * squared distances are compared; sqrt is taken once, at the end;
      * events are visited in x order and the inner loop stops as soon as the
        x-gap alone already exceeds the best distance found so far.
    Ties are resolved to the lexicographically smallest (i, j), matching the
    brute force. Fewer than two events returns (None, None).
    """
    n = len(events)
    if n < 2:
```

```
    return None, None
order = sorted(range(n), key=lambda k: events[k][0])   # indices by x
best_sq, best_pair = math.inf, None
for a in range(n - 1):
    i = order[a]
    xi, yi = events[i]
    for b in range(a + 1, n):
        j = order[b]
        dx = events[j][0] - xi
        dx2 = dx * dx
        if dx2 > best_sq:      # x-gap alone is too big: no later j can win
            break
        dy = events[j][1] - yi
        d2 = dx2 + dy * dy
        pair = (i, j) if i < j else (j, i)
        if d2 < best_sq or (d2 == best_sq and pair < best_pair):
            best_sq, best_pair = d2, pair
return best_pair, math.sqrt(best_sq)
```

Listing 2 – `test_sentinelops.py` (covers every function, the sample data and each boundary case from Part (b); the random tests compare each optimised function with its corrected baseline):

```
"""Test harness for sentinelops.py  -  run:  python -m unittest -v test_sentinelops"""

import random

import unittest


from sentinelops import (

    sort_alerts, sort_alerts_insertion, build_blocklist, is_blocked,

    find_signature, closest_logins, closest_logins_bruteforce,

)


ALERTS = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]

BLOCKLIST = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]

EVENTS = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]


class TestSortAlerts(unittest.TestCase):

    EXPECTED = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]


    def test_sample(self):

        self.assertEqual(sort_alerts(list(ALERTS)), self.EXPECTED)

        self.assertEqual(sort_alerts_insertion(list(ALERTS)), self.EXPECTED)
```

```
def test_no_alert_lost_or_duplicated(self):
    self.assertCountEqual(sort_alerts(list(ALERTS)), ALERTS)

def test_equal_severity_keeps_arrival_order(self):
    data = [("E1", 5), ("E2", 5), ("E3", 5)]
    self.assertEqual(sort_alerts(list(data)), data)
    self.assertEqual(sort_alerts_insertion(list(data)), data)

def test_empty_and_single(self):
    self.assertEqual(sort_alerts([]), [])
    self.assertEqual(sort_alerts([("X", 1)]), [("X", 1)])

def test_already_sorted_and_reversed(self):
    desc = [("a", 9), ("b", 7), ("c", 2)]
    self.assertEqual(sort_alerts(list(desc)), desc)
    self.assertEqual(sort_alerts(list(reversed(desc))), desc)

def test_random_matches_baseline(self):
    rng = random.Random(1)
    for _ in range(200):
```

```
            data = [(f"a{k}", rng.randint(1, 6)) for k in range(rng.randint(0, 40))]
            self.assertEqual(sort_alerts(list(data)), sort_alerts_insertion(list(data)))


class TestIsBlocked(unittest.TestCase):
    def test_sample_list_and_set(self):
        for bl in (BLOCKLIST, build_blocklist(BLOCKLIST)):
            self.assertTrue(is_blocked(bl, "10.0.0.9"))
            self.assertTrue(is_blocked(bl, "10.0.0.5"))
            self.assertFalse(is_blocked(bl, "8.8.8.8"))

    def test_first_middle_last(self):
        for ip in BLOCKLIST:
            self.assertTrue(is_blocked(BLOCKLIST, ip))

    def test_empty_blocklist(self):
        self.assertFalse(is_blocked([], "1.1.1.1"))
        self.assertFalse(is_blocked(build_blocklist([]), "1.1.1.1"))

class TestFindSignature(unittest.TestCase):
    def test_sample(self):
```

```
    self.assertEqual(find_signature("FAILFAIXFAIL", "FAIL"), [0, 8])

def test_pattern_at_very_end(self):
    self.assertEqual(find_signature("xxFAIL", "FAIL"), [2])

def test_pattern_at_very_start(self):
    self.assertEqual(find_signature("FAILxx", "FAIL"), [0])

def test_no_false_hit_when_last_char_differs(self):
    self.assertEqual(find_signature("FAIX", "FAIL"), [])

def test_pattern_longer_than_log(self):
    self.assertEqual(find_signature("ab", "abcd"), [])

def test_pattern_equals_log(self):
    self.assertEqual(find_signature("FAIL", "FAIL"), [0])

def test_overlapping_matches(self):
    self.assertEqual(find_signature("aaaa", "aa"), [0, 1, 2])

def test_empty_log_and_empty_pattern(self):
```

```
        self.assertEqual(find_signature("", "FAIL"), [])
        self.assertEqual(find_signature("abc", ""), [])

    def test_matches_str_find_on_random_text(self):
        rng = random.Random(2)
        for _ in range(300):
            log = "".join(rng.choice("ab") for _ in range(rng.randint(0, 30)))
            pat = "".join(rng.choice("ab") for _ in range(rng.randint(1, 4)))
            expected = [i for i in range(len(log) - len(pat) + 1) if log.startswith(pat, i)]
            self.assertEqual(find_signature(log, pat), expected)


class TestClosestLogins(unittest.TestCase):
    def test_sample(self):
        pair, dist = closest_logins(EVENTS)
        self.assertEqual(pair, (0, 5))
        self.assertAlmostEqual(dist, 2 ** 0.5)
        self.assertEqual(closest_logins_bruteforce(EVENTS), (pair, dist))

    def test_fewer_than_two_events(self):
        self.assertEqual(closest_logins([]), (None, None))
```

```
    self.assertEqual(closest_logins([(1, 1)]), (None, None))

def test_exactly_two_events(self):
    self.assertEqual(closest_logins([(0, 0), (3, 4)]), ((0, 1), 5.0))

def test_duplicate_locations_give_zero(self):
    self.assertEqual(closest_logins([(1, 1), (9, 9), (1, 1)]), ((0, 2), 0.0))

def test_same_x_vertical_line(self):
    self.assertEqual(closest_logins([(5, 0), (5, 10), (5, 11)]), ((1, 2), 1.0))

def test_ties_match_bruteforce(self):
    ev = [(0, 0), (1, 0), (5, 5), (6, 5), (0, 1)]       # several pairs at distance 1
    self.assertEqual(closest_logins(ev), closest_logins_bruteforce(ev))

def test_random_matches_bruteforce(self):
    rng = random.Random(3)
    for _ in range(300):
        ev = [(rng.randint(0, 25), rng.randint(0, 25)) for _ in range(rng.randint(2, 40))]
        self.assertEqual(closest_logins(ev), closest_logins_bruteforce(ev))
```

```
if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Result of the test run (`python -m unittest test_sentinelops`):

```
.........................

----------------------------------------------------------------------

Ran 25 tests in 0.53s


OK
```

Table 13 – Code-quality checklist

Requirement	Where it is met
Docstrings	Module and every function in sentinelops.py
Meaningful comments	Inline notes on the pruning test, the for … else, and the guards
Test harness checking every function	test_sentinelops.py: 25 tests across the four functions plus build_blocklist
Boundary cases from (b)	Empty blocklist, pattern at the end, pattern longer than the log, equal severities, fewer than two events

Output unchanged by optimisation	Random tests compare each optimised function with its corrected baseline

Code of Conduct

I, M. Rumana Khan (Reg. No: 192471025), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.RUMANA KHAN **Date:** 20-10-2026